

Heap Sort

Last Updated : 19 Aug, 2025



Heap sort is a comparison-based sorting technique based on [Binary Heap Data Structure](#). It can be seen as an optimization over [selection sort](#) where we first find the max (or min) element and swap it with the last (or first). We repeat the same process for the remaining elements. In Heap Sort, we use Binary Heap so that we can quickly find and move the max element in $O(\log n)$ instead of $O(n)$ and hence achieve the $O(n \log n)$ time complexity.

Heap Sort Algorithm

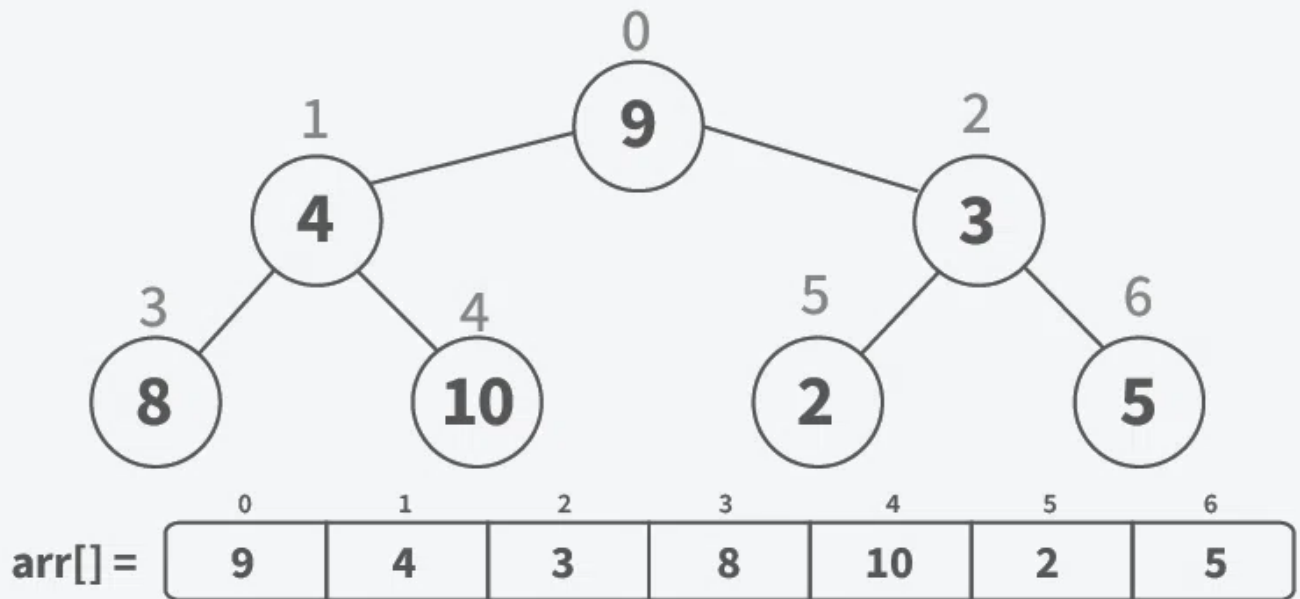
First convert the array into a [max heap](#) using **heapify**. Please note that this happens in-place. The array elements are re-arranged to follow heap properties. Then one by one delete the root node of the Max-heap and replace it with the last node and **heapify**. Repeat this process while size of heap is greater than 1.

- Rearrange array elements so that they form a Max Heap.
- Repeat the following steps until the heap contains only one element:
 - Swap the root element of the heap (which is the largest element in current heap) with the last element of the heap.
 - Remove the last element of the heap (which is now in the correct position). We mainly reduce heap size and do not remove element from the actual array.
 - Heapify the remaining elements of the heap.
- Finally we get sorted array.

Detailed Working of Heap Sort

Step 1: Treat the Array as a Complete Binary Tree

We first need to visualize the array as a **complete binary tree**. For an array of size n , the root is at index 0, the left child of an element at index i is at $2i + 1$, and the right child is at $2i + 2$.

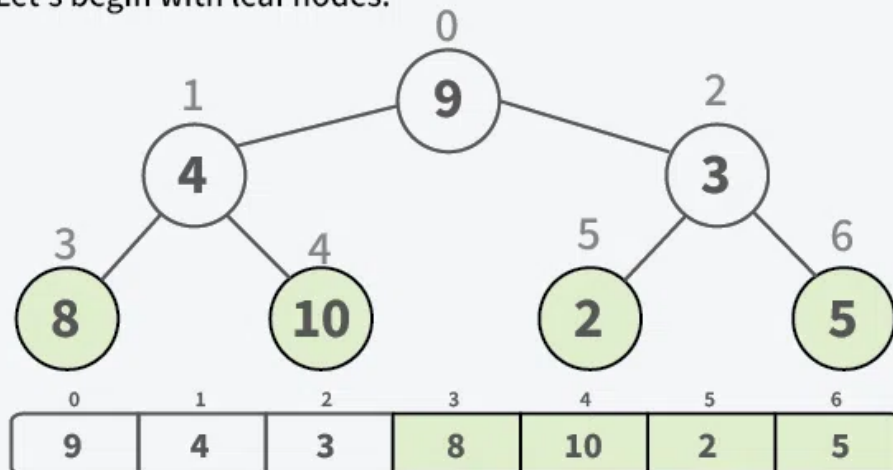


Visualize the Array as a Complete Binary Tree

Step 2: Build a Max Heap

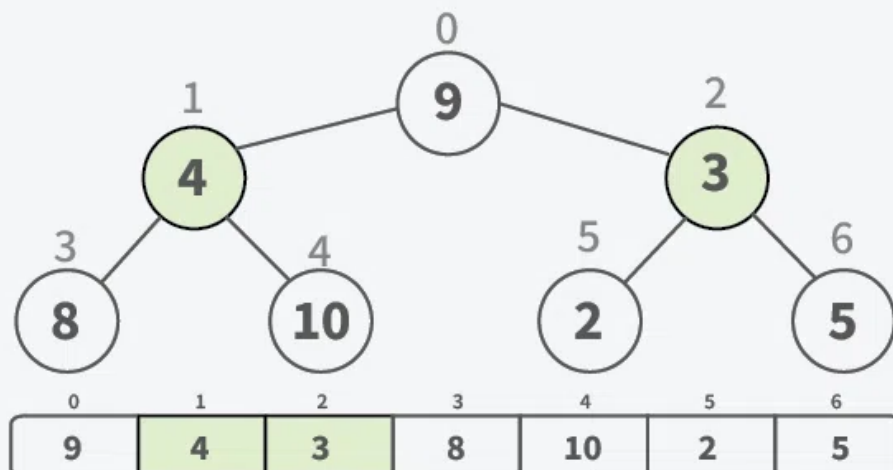
01
Step

Compare each node with its children, ensuring parent nodes are larger. This causes smaller nodes to bubble down and larger nodes to rise to the top. Let's begin with leaf nodes.



02
Step

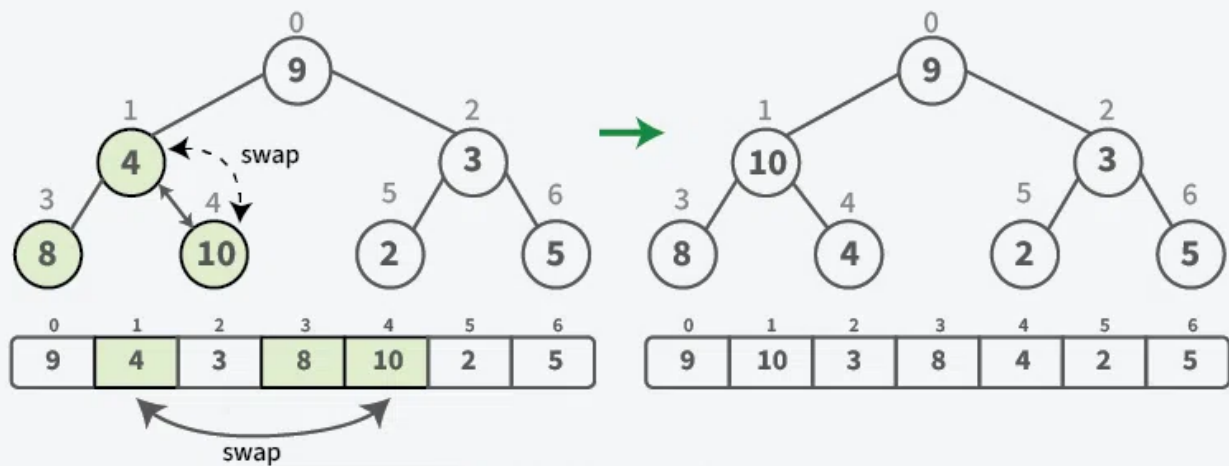
Let's look in the next upper level (node 4 and 3)



Heapify Binary Tree

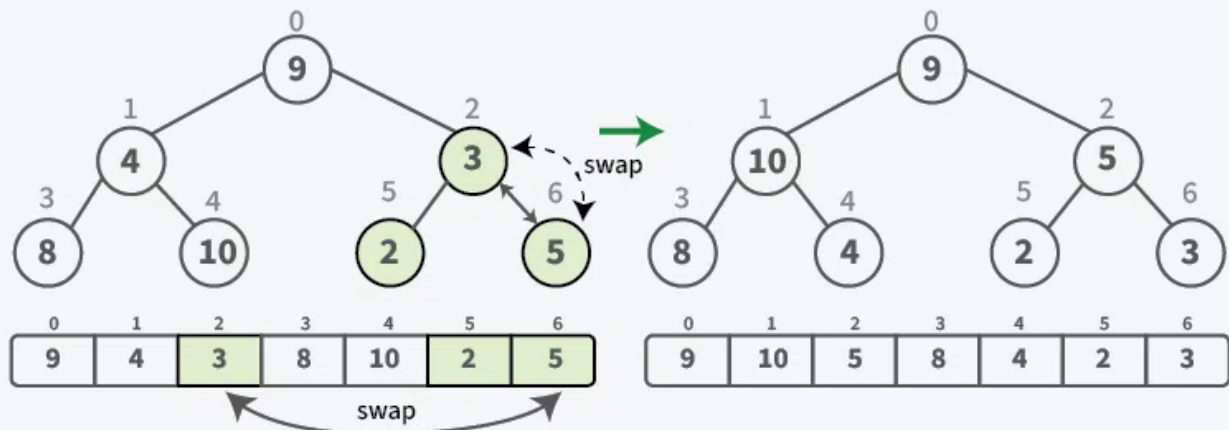
03
Step

Node 4 is smaller than its child node (10), so swap it with the larger child to maintain the property that the parent should be larger than its children.



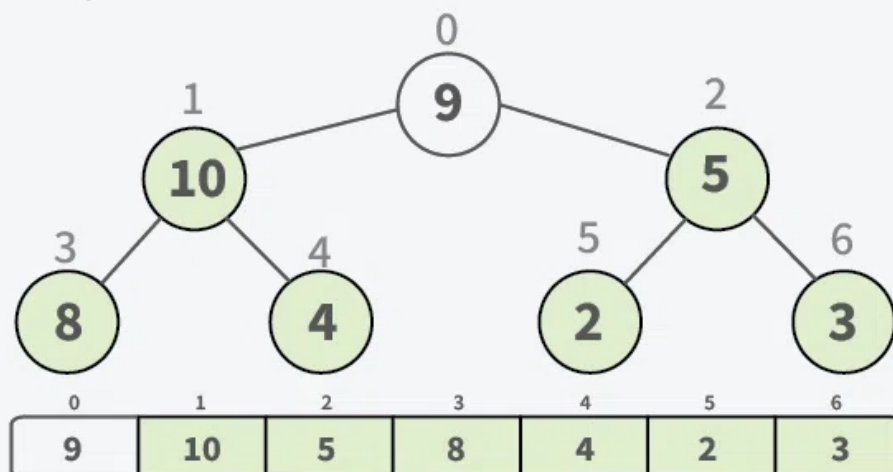
04
Step

Check for the next node (3) in current level. Since, it has a larger child, so swap it to ensure the parent has a larger value.



05
Step

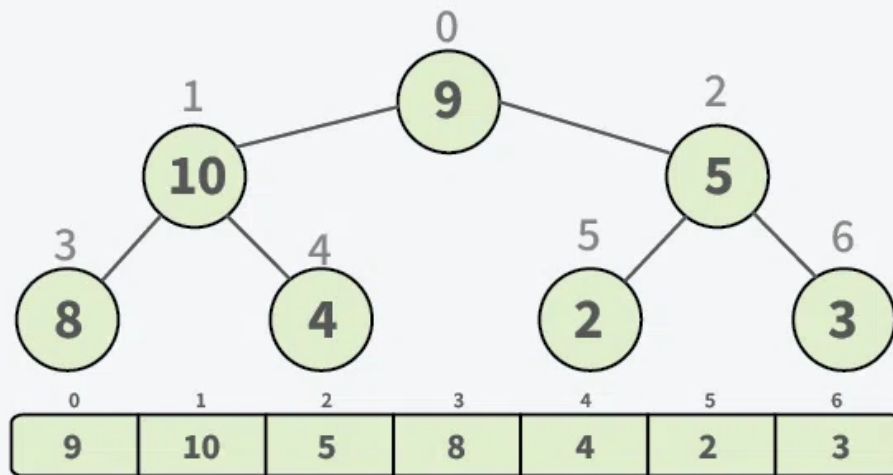
After the completion of current level, we have now two smaller valid max heap



Heapify Binary Tree

06
Step

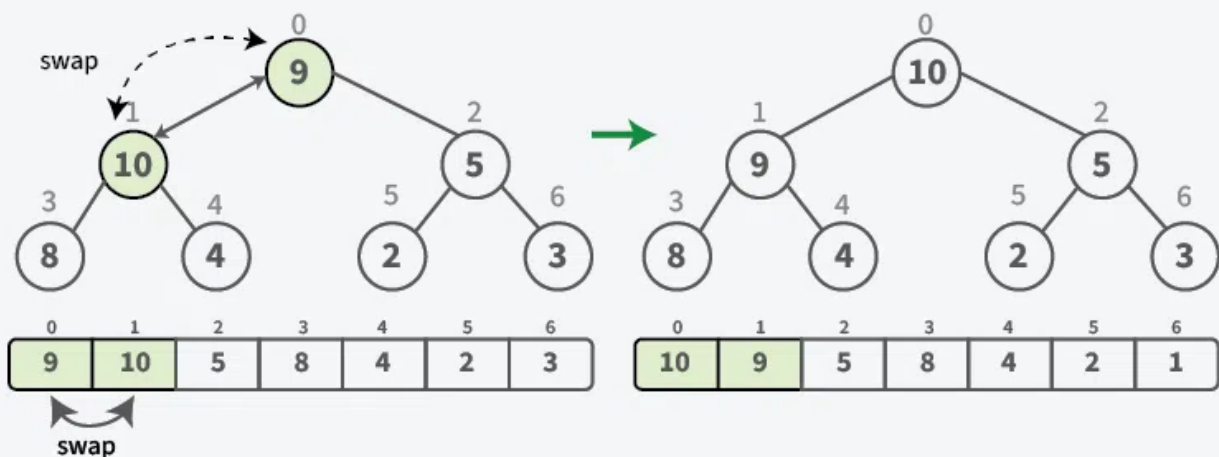
Let's move to the next upper level. Here we've node 9 at the root.



Heapify Binary Tree

07
Step

Node 9 is smaller than its child node (10), so swap it with the larger child to maintain the property that the parent should be larger than its children.



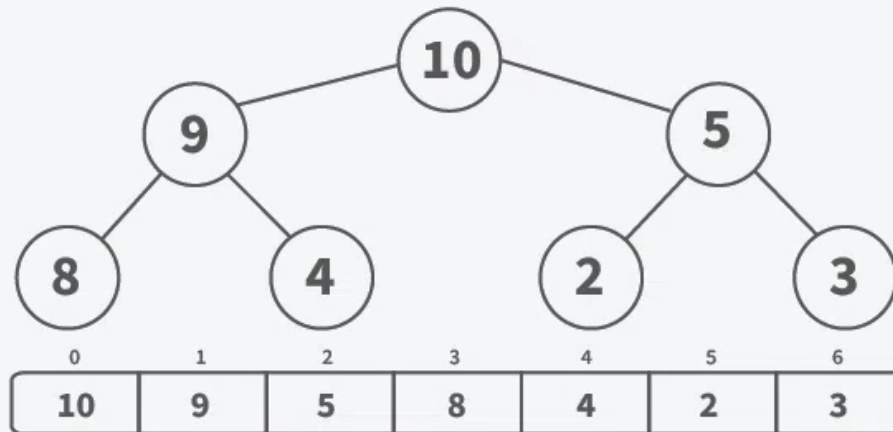
Heapify Binary Tree

Step 3: Sort the array by placing largest element at end of unsorted array.



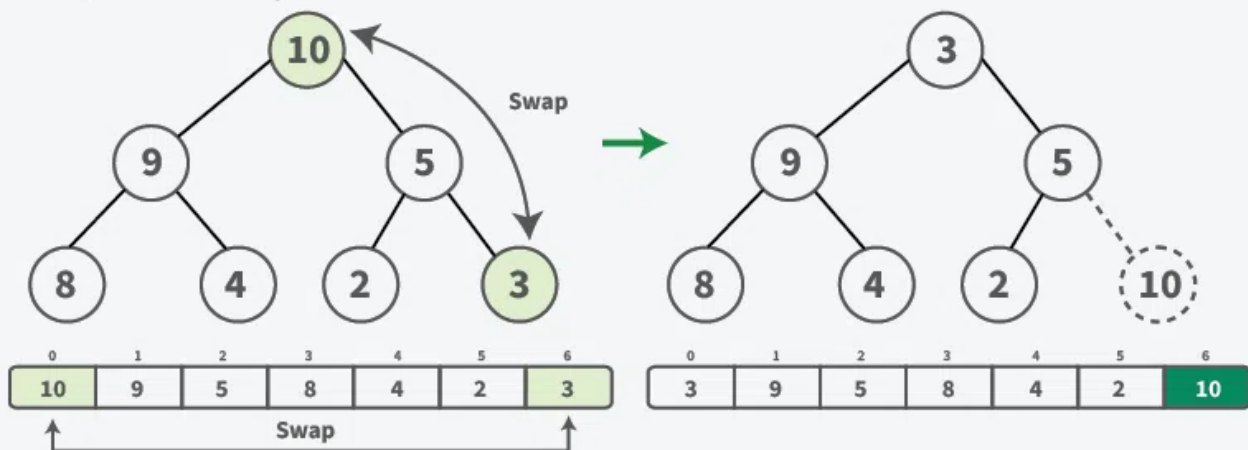
01
Step

Let's assume we have transformed the given array to follow the max heap property. Here's how our array would look in max heap form.



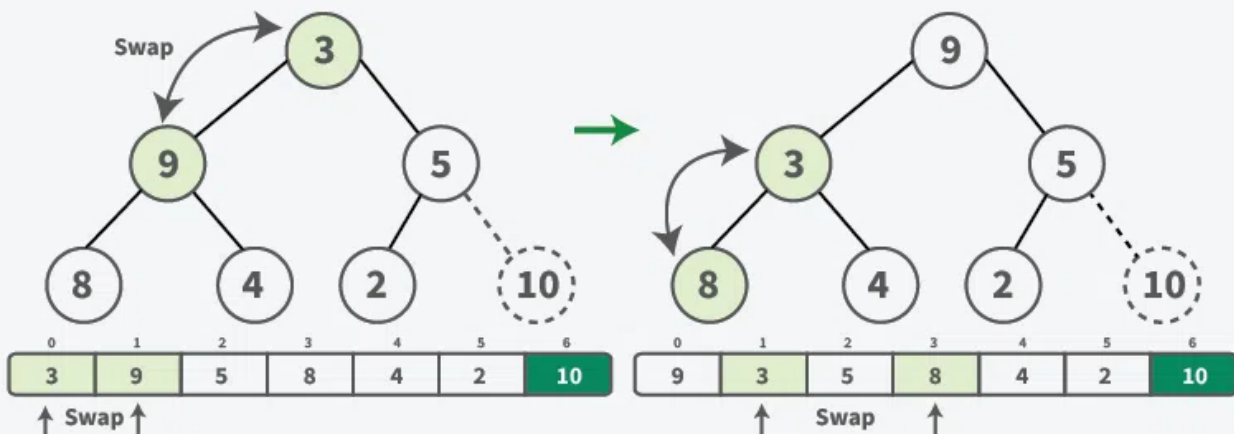
02
Step

Swap the maximum element (10) with the last element (3) in the unsorted array. Decrease the size of the heap by one (ignore the last element, as it is now sorted).



03
Step

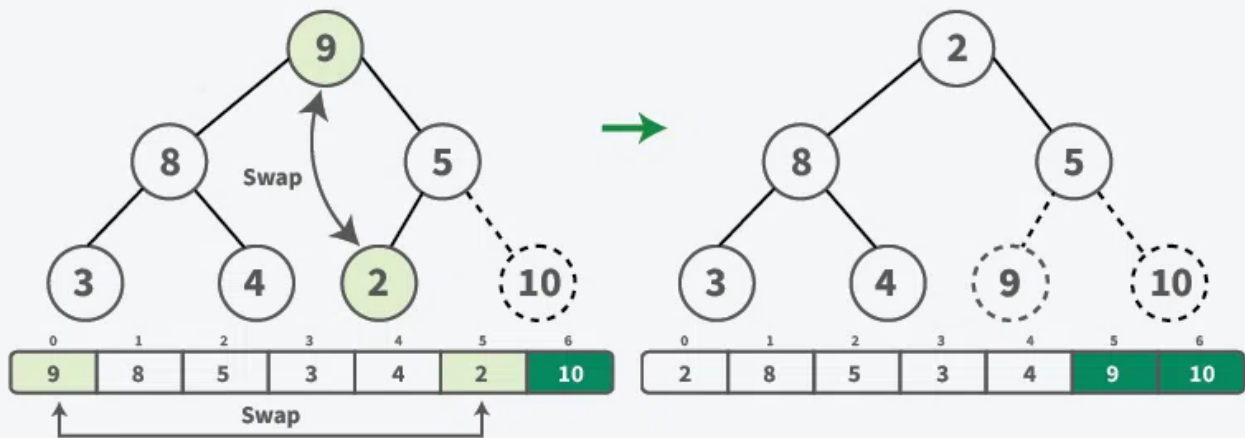
Now, root violate the max-heap property, So, heapify it. Swap node 3 with its largest child (9). Still node 3 have larger children, so swap it with largest one (node 8).



Remove from Max Heap

04
Step

Now, we have a max heap. Swap the maximum element (9) with the last element (2) in the unsorted array, then decrease the heap size by one (ignoring the second last element as it's now sorted).

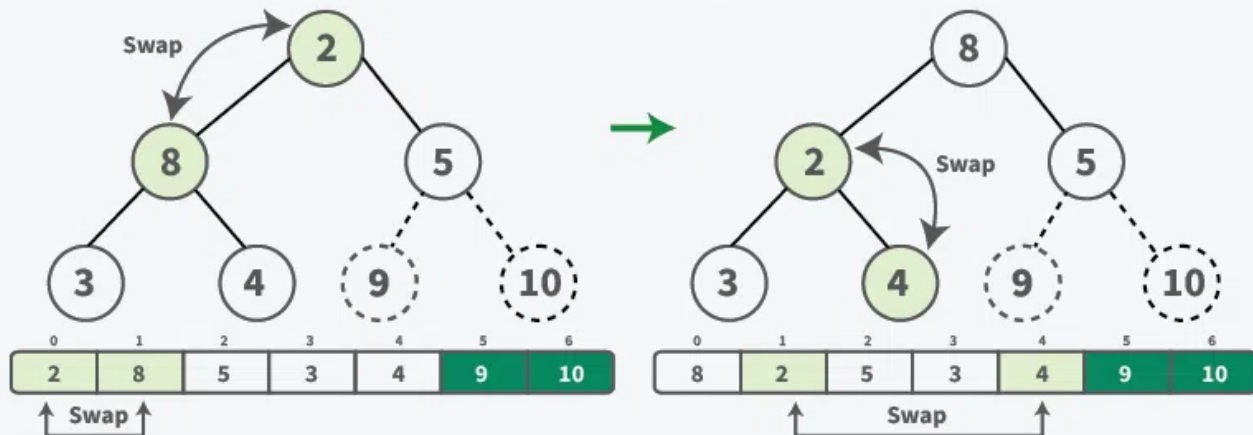


05
Step

Now, root violate the max-heap property, So, heapify it.

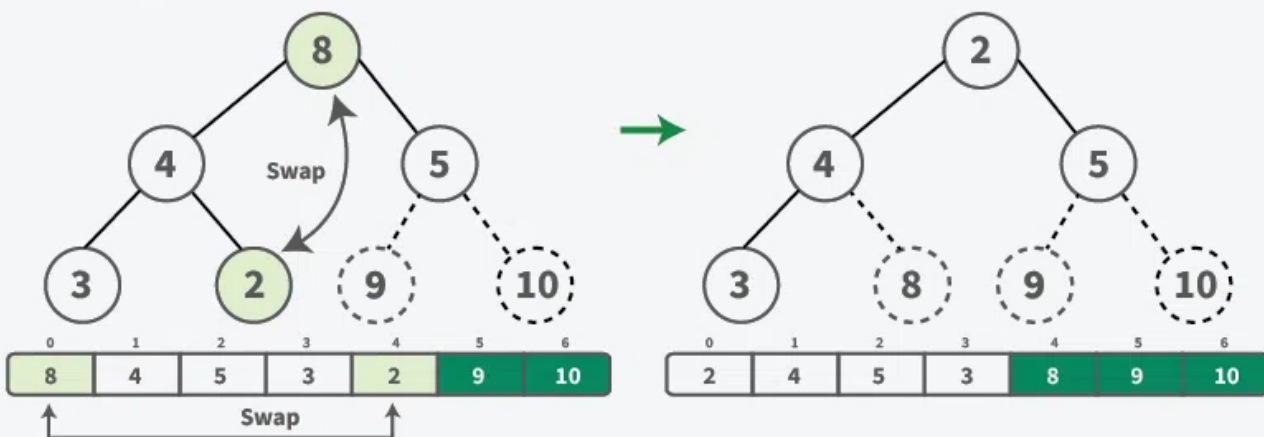
Swap node 2 with its largest child (8).

Still node 2 have larger children, so swap it with largest one (node 4)



06
Step

Now, we have a max heap. Swap the maximum element (8) with the last element (2) in the unsorted array, then decrease the heap size by one (ignoring the third last element as it's now sorted).



Remove from Max Heap

In the illustration above, we have shown some steps to sort the array. We need to keep repeating these steps until there's only one element left in the heap.

C++

C

Java

Python

C#

JavaScript

PHP

// C++ program for implementation of Heap Sort using vector



```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// To heapify a subtree rooted with node i
```

```
// which is an index in arr[].
```

```
void heapify(vector<int>& arr, int n, int i){
```

```
    // Initialize largest as root
```

```
    int largest = i;
```

```
    // left index = 2*i + 1
```

```
    int l = 2 * i + 1;
```

```
    // right index = 2*i + 2
```

```
    int r = 2 * i + 2;
```

```
    // If left child is larger than root
```

```
    if (l < n && arr[l] > arr[largest])
```

```
        largest = l;
```

```
    // If right child is larger than largest so far
```

```
    if (r < n && arr[r] > arr[largest])
```

```
        largest = r;
```

```
    // If largest is not root
```

```

    if (largest != i) {
        swap(arr[i], arr[largest]);

        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}

// Main function to do heap sort
void heapSort(vector<int>& arr){
    int n = arr.size();

    // Build heap (rearrange vector)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // One by one extract an element from heap
    for (int i = n - 1; i > 0; i--) {

        // Move current root to end
        swap(arr[0], arr[i]);

        // Call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}

// A utility function to print vector of size n
void printArray(vector<int>& arr){
    for (int i = 0; i < arr.size(); ++i)
        cout << arr[i] << " ";
    cout << "\n";
}

// Driver's code
int main(){
    vector<int> arr = { 9, 4, 3, 8, 10, 2, 5 };

    // Function call
    heapSort(arr);

    cout << "Sorted array is \n";
}

```



```
    printArray(arr);  
}
```

Output

```
Sorted array is  
2 3 4 5 8 9 10
```

Complexity Analysis of Heap Sort

Time Complexity: $O(n \log n)$

Auxiliary Space: $O(\log n)$, due to the recursive call stack. However, auxiliary space can be $O(1)$ for iterative implementation.

Important points about Heap Sort

- An in-place algorithm.
- Its typical implementation is not stable but can be made stable (See [this](#))
- Typically 2-3 times slower than well-implemented [QuickSort](#). The reason for slowness is a lack of locality of reference.

Advantages of Heap Sort

- **Efficient Time Complexity:** Heap Sort has a time complexity of $O(n \log n)$ in all cases. This makes it efficient for sorting large datasets. The **log n factor comes from the height of the binary heap**, and it ensures that the algorithm maintains good performance even with a large number of elements.
- **Memory Usage:** Memory usage can be minimal (by writing an iterative `heapify()` instead of a recursive one). So apart from what is necessary to hold the initial list of items to be sorted, it needs no additional memory space to work
- **Simplicity:** It is simpler to understand than other equally efficient sorting algorithms because it does not use advanced computer science concepts such as recursion.

Disadvantages of Heap Sort

- **Costly:** Heap sort is costly as the constants are higher compared to merge sort even if the time complexity is $O(n \log n)$ for both.
- **Unstable:** Heap sort is unstable. It might rearrange the relative order.

- **Inefficient:** Heap Sort is not very efficient because of the high constants in the time complexity.

Heap Sort - Data Structures and Algorithms

 Comment

More info 

Advertise with us

Similar Reads

Basics & Prerequisites 

Data Structures 

Algorithms 

Advanced 

Interview Preparation 

Practice Problem 

Do Not Sell or Share My Personal Information